

Object Outline Detector: Hybrid Deep-Learning and Classical Computer-Vision Pipeline for Interactive Object Outlining

Aritra Dhan | 24BAI10513 | aritra.24bai10513@vitbhopal.ac.in

September 2026

Abstract

This report describes the design and implementation of an object outlining tool that combines a deep-learning instance-segmentation model (YOLO11) with a classical computer-vision fallback pipeline. Given an input image and a natural-language object name, the system produces a clean visual outline of the requested object, along with supporting edge, line, and corner analysis (Canny, Hough, Harris, and Hessian). The project was initially prototyped in a Jupyter/Colab notebook and was subsequently refactored into a standalone, command-line-executable Python application to satisfy reproducibility and deployment requirements. This report covers the motivation, methodology, implementation details, results, and conclusions of the project.

1. Introduction

Object outlining — isolating and highlighting a specific object within an image — is a common preprocessing step in applications such as photo editing, augmented reality, product cataloguing, and dataset annotation. Modern deep-learning segmentation models such as YOLO can identify a fixed vocabulary of common object classes with high accuracy, but real-world images often contain objects outside that vocabulary, or the model may fail to detect an object with sufficient confidence. This motivates a hybrid approach: rely on a trained segmentation model when possible, and fall back to a robust, model-free classical vision pipeline when it is not.

The goal of this project was to build such a hybrid system that (1) accepts an image and an object name as input, (2) attempts segmentation using YOLO11, (3) falls back to a Canny-edge-and-contour based structural pipeline if the object is not recognized, and (4) presents the result in multiple visual styles, together with quantitative image-structure statistics.

2. Objectives

- Detect and segment a user-specified object in an arbitrary input image.
- Provide a reliable fallback mechanism for objects outside the trained model's class vocabulary.
- Generate multiple presentation styles of the detected outline (outline overlay, sketch, silhouette).
- Compute supporting classical computer-vision statistics (edges, lines, corners, interest points) for analysis.
- Package the system as a reproducible, command-line-executable application rather than a notebook-only demo.

3. Methodology

The pipeline consists of the following stages, summarized in Table 1 below.

Stage	Technique	Purpose
Primary detection	YOLO11 instance segmentation (yolo11n-seg)	Pixel-accurate mask for the requested object class
Fallback detection	Canny edges + morphological closing + contour extraction	Outline generation when the object is outside YOLO's class list
Mask refinement	Morphological close/open	Removes noise while preserving object instances

Supporting analysis	Hough Line Transform, Harris Corner Detector, Hessian interest points	Quantifies structural / geometric content of the image
Output rendering	Contour drawing (green outline, sketch, silhouette)	Produces four user-selectable visual styles

Table 1: Pipeline stages, techniques used, and their purpose.

3.1 Segmentation

The image is passed to a pretrained YOLO11 segmentation model (yolo11n-seg). The user-provided object name is normalized (lower-cased, alias-mapped — e.g. “bike” to “bicycle”) and matched against the model’s class labels. If a match is found, all predicted instances of that class are merged into a single binary mask using a pixel-wise maximum operation, and the mask is cleaned using morphological closing followed by opening to remove small noise artifacts while preserving all detected instances.

3.2 Structural Fallback

When YOLO does not return a mask for the requested class (either because the object is outside its 80-class vocabulary or was not detected with sufficient confidence), the system falls back to a classical pipeline: the image is converted to grayscale, blurred with a Gaussian filter, and passed through an automatically-thresholded Canny edge detector (threshold derived from the image’s median intensity). The resulting edge map is closed morphologically to bridge small gaps, and external contours are extracted and filtered by area, retaining up to the eight largest structurally significant regions.

3.3 Supporting Classical CV Analysis

Independent of the outline itself, the system computes: (a) Canny edges over the whole image; (b) straight line segments via the Probabilistic Hough Line Transform; (c) corner points via the Harris Corner Detector; and (d) interest points via a Hessian-matrix determinant response with non-maximum suppression. These are reported as counts and visualized as overlay images, giving a quantitative sense of the structural complexity of the scene.

3.4 Output Rendering

From the final set of contours (whichever source produced them), four visual styles are rendered: a green outline drawn over the original image, a black-on-white sketch, a filled white silhouette on a black background, and an original-plus-outline composite. The user may select one style as the primary result or request all four simultaneously.

4. Implementation

The project was implemented in Python 3, using OpenCV for image processing, NumPy for array operations, and the Ultralytics YOLO11 implementation for segmentation. The original prototype was developed and iterated on in a Google Colab notebook, using Colab-specific utilities for file upload/download and an ipywidgets dropdown for interactive style selection.

To satisfy the requirement that the project be fully executable from the command line, the notebook was refactored into a modular Python package: segmentation logic, classical CV routines, outline/style rendering, and reporting were separated into dedicated modules under `src/`, and a `main.py` script exposes the entire pipeline through command-line arguments (input image path, target object name, output style, confidence threshold, model choice, and output directory). Colab-specific calls (`files.upload`, `files.download`, `ipywidgets`) were replaced with standard file I/O and `argparse`-based flags, with no change to the underlying computer-vision logic.

5. Results

The system successfully distinguishes between two operating modes depending on whether the requested object falls within YOLO's trained class vocabulary: an “AI segmentation” mode producing pixel-accurate masks for known classes (e.g. person, car, dog, bicycle), and a “structural fallback” mode producing contour-based outlines for arbitrary or unrecognized objects. For each processed image, the tool reports the processing mode used, the number of contours retained, object pixel area and coverage percentage, boundary pixel count, and the counts of detected Hough lines, Harris corners, and Hessian interest points. All eight intermediate/final images plus a zipped results archive are written to disk for inspection, alongside four selectable output-style renderings.

Qualitatively, the AI segmentation path produces tighter, more accurate outlines for its known object classes, while the structural fallback reliably produces a usable (if coarser) outline for arbitrary objects, confirming the value of the hybrid design over either approach alone.



6. Challenges and Limitations

- YOLO's segmentation is limited to its trained class vocabulary (COCO's 80 classes); the fallback path, while robust, cannot guarantee the outline corresponds to the specific object the user intended when multiple structural regions are present.
- Detection quality depends on the chosen confidence threshold; a threshold that is too low may pick up false positives, while too high a threshold may miss valid detections.
- Porting away from the Colab environment required removing interactive/GUI-dependent components (file upload dialog, dropdown widget) and replacing them with equivalent command-line arguments, without altering the underlying vision algorithms.

7. Conclusion

This project demonstrates a practical hybrid approach to object outlining that combines the accuracy of a modern deep-learning segmentation model with the robustness and generality of classical computer-vision techniques. The resulting command-line tool accepts an arbitrary image and object description, gracefully falls back when the trained

model cannot help, and produces multiple useful visual and quantitative outputs. Future work could extend the class vocabulary through fine-tuning or open-vocabulary segmentation models, and could improve the fallback path with more targeted region-of-interest selection based on the object name (e.g. using text-to-region models).

References

Ultralytics YOLO11 documentation: <https://docs.ultralytics.com>

OpenCV documentation: <https://docs.opencv.org>

Canny, J. (1986). A Computational Approach to Edge Detection. IEEE Transactions on Pattern Analysis and Machine Intelligence.

Harris, C., & Stephens, M. (1988). A Combined Corner and Edge Detector. Proceedings of the 4th Alvey Vision Conference.